

On Transforming Spatial Geometry into Volumetric Forms Following Bézier Trajectories

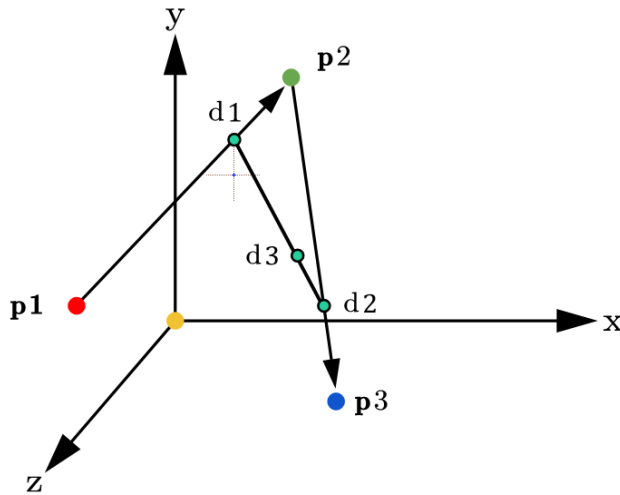
By UkawaJun

Abstract

The core methodology involves the establishment of four distinct mathematical models, implemented using the **HLSL** (High-Level Shading Language) programming language. During the execution of the process function, data for three control points (p_1, p_2, p_3) is transmitted in real-time to the shader for parallel computation.

1. Quadratic Bézier Curve Model

The foundation of the system is a Quadratic Bézier model, which serves as the central path for geometry generation.



$$t = \frac{d1p1}{p1p2} = \frac{d2p2}{p2p3} = \frac{d1d3}{d1d2} \{t \in [0, 1]\}$$

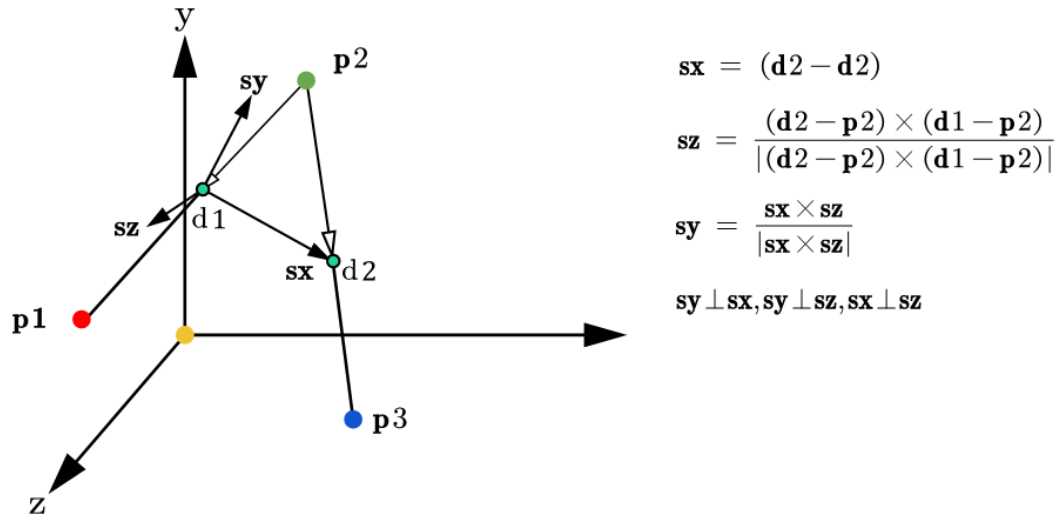
$$P_n = \begin{pmatrix} x \\ y \\ z \end{pmatrix}^{-1} \{x, y, z \in \mathbf{R}\}$$

$$P_n = \text{Bezier}(t)$$

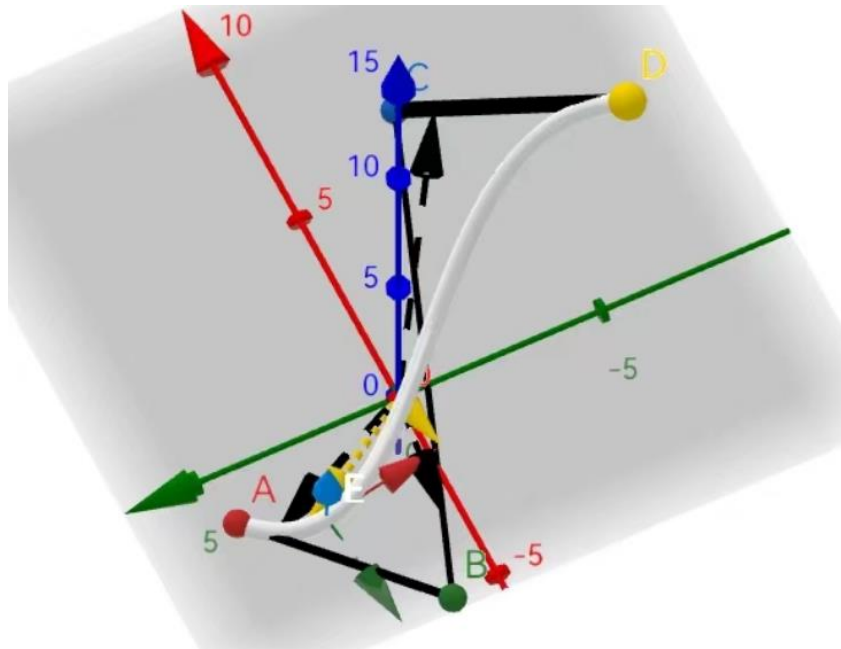
Graph 1 2-dim Bézier Curve Model

2. Construction of the Dynamic Coordinate System

To define the trajectory of the volumetric deformation, a dynamic local reference frame (Moving Frame) is established. This is achieved through vector subtraction and double cross-products, ensuring the geometry aligns correctly with the curve's curvature.



Graph 2 Construction of the Dynamic Coordinate System

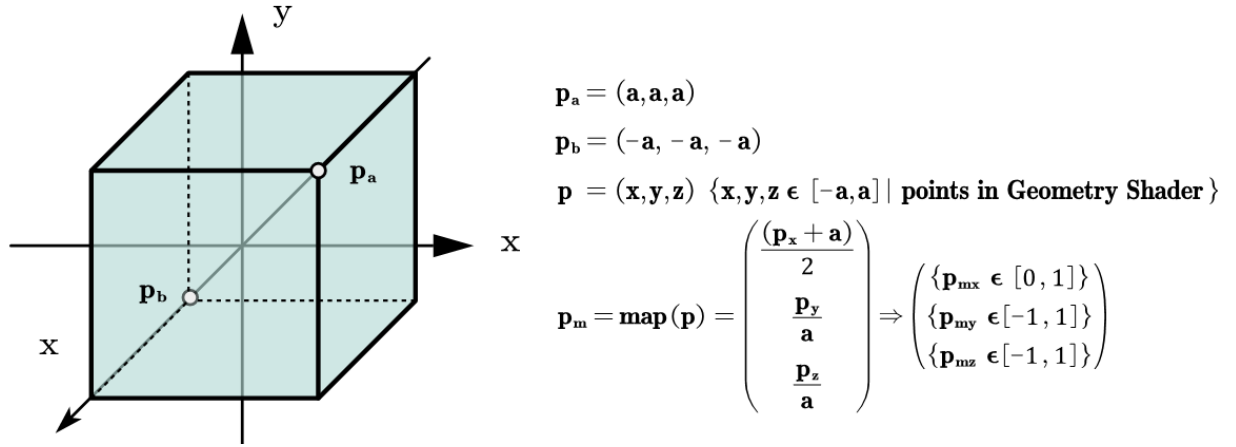


Graph 3 Simulation in Geogebra

3. Volumetric Mapping Algorithm

The mapping algorithm transforms the vertex data of an original cube model into coordinates that conform to the Bézier curve.

1. The X-coordinate of the mesh is mapped to the Bézier parameter t ($0 \leq t \leq 1$).
2. The Y and Z coordinates are normalized to a range between -1 and 1 , representing the cross-sectional thickness and offset data relative to the curve's center.

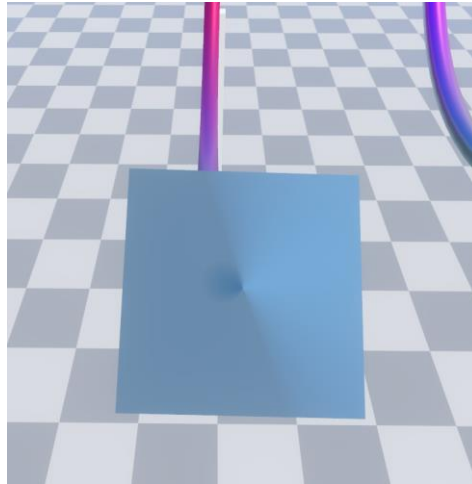


Graph 4 Mapping model

4. Cross-Sectional Analysis

As shown in the cross-sectional view, the trajectory of the curve is primarily driven by the mapped x -axis, while the y and z planes define the volumetric "shell" around the path.

This figure shows the cross-section of the curve, from which it can be seen that the curve trajectory is mainly along the x -axis:



Graph 5 cross-section of the curve

5. Final Transformation Logic

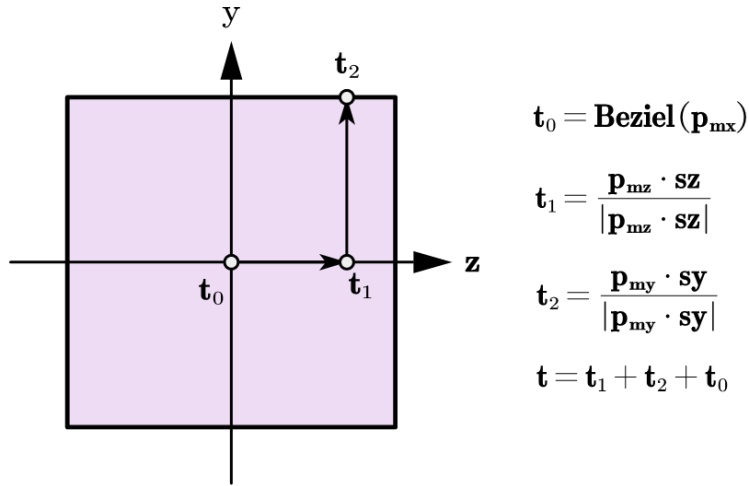
The final implementation of the algorithm follows a multi-step transformation:

t_0 : Calculate the central point of the curve diameter (the spine).

t_1 : Apply the offset along the local Z -axis.

t_2 : Apply the offset along the local Y -axis.

The final output point $t = t_0 + t_1 + t_2$ represents the result of the completed spatial transformation.

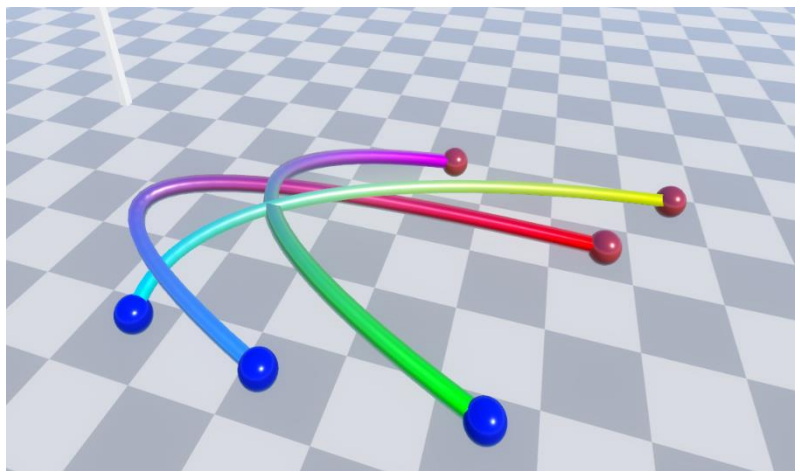


Graph 6 Caculate the offset

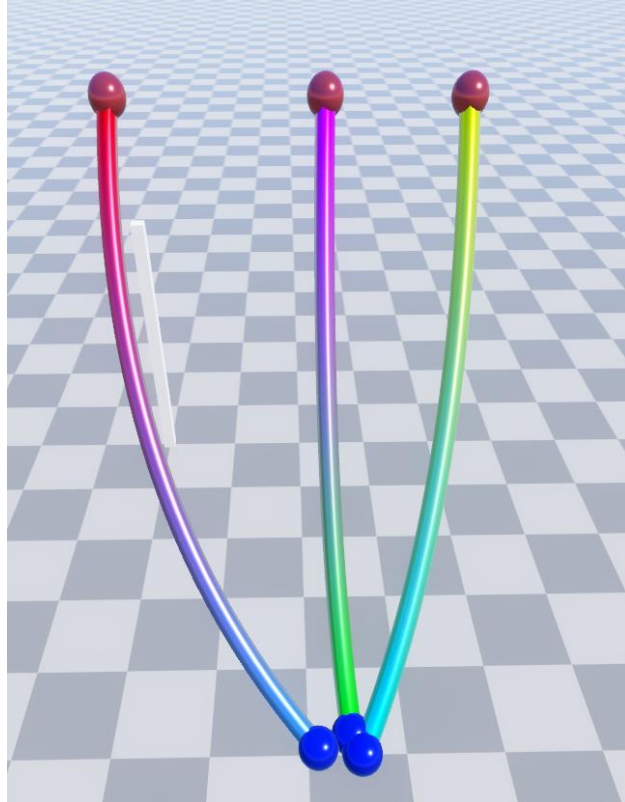
6. Final Result Like:

By leveraging the principle of x as the Bézier parameter t , the system can be modified to achieve advanced procedural effects, including:

1. Variable-width trajectories (e.g., tapered or horn-like structures).
2. Wavy or corrugated shells through periodic functions.
3. Smooth color gradients based on the parameter t .
4. Cubic Bézier integration by extending the mapping logic to handle four control points.

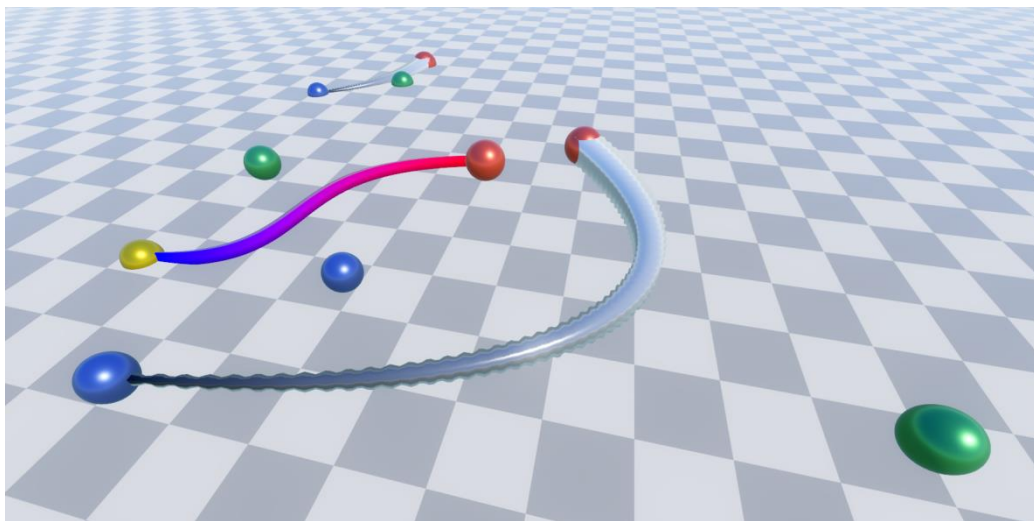


Graph 7 Basic result



Graph 8 Basic result – upright ropes

Additionally, according to the principle of x being the Bézier parameter t , we can also appropriately modify it to achieve effects such as changing the width of the curve trajectory, wavy watch cases, smooth color transitions, and the cubic Bézier curve drawing method modified based on this principle.



Graph 9 Special ropes

Copyright & License Declaration Copyright © 2026 UkawaJun. All rights reserved. This document is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0). The source code associated with this project is available under the MIT License.